

The Decision Boundary.

An Exploratory Evaluation of LLM Autonomy in Research-Funding Eligibility Workflows

Author: Hanisha Pulimaddi

Date: 3 October 2026

1. Abstract

Large language models (LLMs) are increasingly used for rule-based administrative decisions, such as research-funding eligibility. This study compares how different allocations of decision-making between an LLM and deterministic software affect accuracy and auditability. Three architectures were tested on 20 synthetic applicant profiles: (A) LLM-only decision-making, (B) LLM decision-making with JSON validation, and (C) LLM fact extraction followed by deterministic rule execution. Using one model (openai/gpt-oss-20b via Groq) at temperature 0 in a single run of 60 calls, A and B were correct on 20/20 cases and C on 19/20, for both eligibility and effective eligibility date. The single error arose in C's extraction step, where the model asserted an unsupported fact; the rule engine applied that fact correctly.

Accuracy therefore did not meaningfully distinguish the architectures. The key difference was auditability: C decisions could be replayed and its error localised, while A and B lacked an independently recomputable decision path. They reframe the question from whether an LLM can reach the correct decision to where errors should be allowed to live within an AI-assisted administrative workflow.

2. Introduction

Research administration involves many high-volume decisions governed by written rules, such as eligibility windows, career interruptions and allowable expenses [1]. These decisions require both natural-language interpretation of applicant information and formal rule application using thresholds, conditions and calculations.

LLMs are well suited to interpreting unstructured information, although they can produce fluent but unsupported content [5], and their explanations do not always reflect how an answer was reached [4]. This raises a key design question: how much of the decision should the model own? Research on human interaction with automation treats the level of automation at each stage of a decision as a design choice [6]. One approach gives the LLM full decision-making authority; another limits it to extracting structured facts while conventional software applies the rules. Hybrid approaches fall between these extremes.

This report compares three such architectures on a single eligibility task. Rather than identifying a "best" architecture, the experiment examines where errors occur and how easily decisions can be inspected, reproduced and traced after the fact, properties argued to be essential for high-stakes and accountable decision systems [7], [8].

3. Background and Motivation

The task is based on the ARC DECRA eligibility framework, where eligibility depends on PhD conferral dates and allowable career interruptions [1]. The process involves two forms of reasoning: interpreting applicant information and applying formal rules, thresholds and conditions. The three architectures represent different levels of automation [6], with A and B allowing the LLM to make the final decision, while C limits the LLM to extracting facts and uses deterministic software for rule application. Auditability is important because LLMs can produce unsupported information [5], and their explanations may not reflect how an answer was reached [4], making it necessary to understand whether a decision can be inspected, reproduced and traced after the fact [7], [8].

The design boundary comes from the author's previous systems, where LLMs were used for interpretation while deterministic code handled formal decisions. In an automated planning-

compliance checker, the LLM extracted rules while a deterministic engine performed compliance calculations. In WayFind, the LLM was restricted to one stage of a five-stage pipeline, with eligibility and ordering handled by code. This study tests that same engineering principle in a controlled setting to examine how the allocation of responsibility affects accuracy, error traceability and auditability.

4. Research Question and Hypotheses

Research question: How does allocating decision-making between an LLM and deterministic software affect the accuracy and auditability of research-funding eligibility assessment?

Two dimensions are examined: **accuracy**, whether the system produces the correct eligibility decision and effective date; and **auditability**, whether the decision can be reconstructed from explicit intermediate information [8]. The study is exploratory, with non-preregistered working expectations:

- H1 (accuracy): On rule-heavy cases, architectures where the LLM applies the rules (A, B) will make more rule-application errors than deterministic rule execution (C) [5], [6].
- H2 (validity): JSON schema validation (B) will produce machine-readable outputs, but schema validity will not guarantee correct decisions [5].
- H3 (auditability): Structured facts with deterministic execution (C) will allow decisions to be independently recomputed and errors localised to specific inputs or rule steps [7], [8].

5. Methodology

5.1 Dataset

Twenty synthetic applicant profiles (A01-A20) were created to test clear cases, boundary dates, multiple interruptions, unemployment, medical interruptions, relocation caps, concurrent employment and primary-carer periods. Ground truth for eligibility, effective date, claimable months and reasoning was established before testing.

The profiles were then cleaned to remove answer or rule leakage and split into “applicants.json”, containing only IDs and profile text, and “ground_truth.json”, used solely for evaluation. Automated checks confirmed that no ground-truth information appeared in the applicant profiles or prompts. The cleaned dataset was then frozen and used unchanged throughout the experiment.

A10: Dr James Wilson was awarded his PhD on 15 August 2019. After conferral, he had seven months of parental leave followed later by twelve months in a non-research position that was not concurrent with research employment. The periods did not overlap.

5.2 Eligibility rule

The study used a simplified eligibility rule adapted from ARC policy [1], rather than the official wording. An applicant was eligible if their effective PhD award date was on or after **1 March 2021**. The effective date was calculated by extending the PhD award date by allowable post-PhD career interruptions, counted in calendar months, with overlapping periods counted only once.

Allowable interruptions included unemployment, medical conditions, caring responsibilities, parental leave, primary-carer periods (up to two years per dependent child, including parental leave), eligible non-research employment, and international relocation (up to three months per relocation). Limited access to research facilities and disaster recovery were also included.

5.3 Architectures

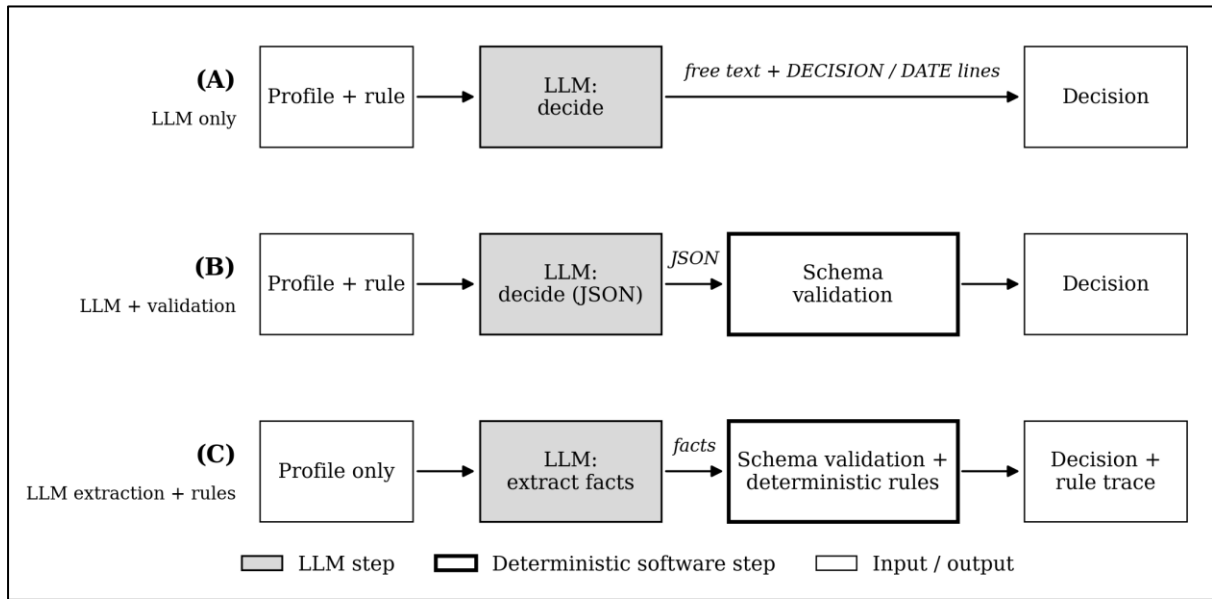


Figure 1. Experiment Architecture Diagram

- **A: LLM only.** The model receives the rule and the profile, explains its reasoning briefly, and ends with `DECISION: ELIGIBLE|INELIGIBLE` and `EFFECTIVE_DATE: YYYY-MM-DD` lines, which are parsed. The model owns the whole process: identifying facts, interpreting interruptions, calculating the effective date, applying the rule and deciding.
- **B: LLM + structured validation.** The model receives the rule and the profile and must return a JSON object with `eligible`, `effective_eligibility_date`, `claimable_interruption_months` and `reasoning`. The object is validated against a strict schema (Pydantic; no extra fields, typed values). The model still makes the decision; the schema checks shape, not substance.
- **C: LLM extraction + deterministic rules.** The model is told: *"You are an information extraction component. Do not determine eligibility. Do not calculate the effective eligibility date. Do not decide whether the applicant is eligible."* It receives only the profile (not the rule) and returns the PhD award date and a list of interruptions, each with a type, duration and attributes (`after_phd`, `concurrent_with_research_employment`, `dependent_children`, `within_primary_carer_period`). After schema validation, a Python rule engine processes the PhD date and interruptions, computes claimable months and the effective date, compares the result with the threshold, and emits the decision with a step-by-step trace.

5.4 Implementation and procedure

Item	Value
Dataset size	20 synthetic applicant profiles (A01-A20)
Ground truth	Established manually before any model run; held separately and never exposed to the model
Model	openai/gpt-oss-20b [2] via the Groq API [3]
Temperature	0
Runs	1 (run ID 20261003T032528Z-6eba21, 3 October 2026, 03:25-03:30 UTC)
Model input	A and B: rule + profile + instructions/schema. C: profile + extraction instructions/schema (no rule)

Item	Value
Total calls	60 (20 x 3); 60 completed, 0 failed, 0 retried
Evaluation method	Exact match against ground truth for eligibility and effective date; output/schema validity; for C, field-level comparison of extracted facts and deterministic replay of each decision

Table 1. Experimental setup.

All three architectures call a provider-independent client interface; the Groq implementation reads the API key and model name from environment variables. Calls used temperature 0, and JSON mode for B and C. Each call was logged with the run identifier, timestamp, model, applicant, architecture, prompt name and version, prompt hash and full prompt text, raw response, parsed output, status and, for C, the rule trace. The API key and ground truth were never logged alongside prompts. Before the experiment, a one-applicant smoke test (AO1 through each architecture) verified connectivity, prompt construction and parsing. The full experiment ran once, without retries.

5.5 Evaluation metrics

- Eligibility accuracy and effective-date accuracy: exact agreement with ground truth over all 20 cases (a failed call would count as incorrect).
- Output/schema validity: A, decision line parsed; B, JSON passed the decision schema; C, JSON passed the facts schema.
- Internal consistency (A, B): whether the stated decision agrees with the model's own effective date; for B, whether PhD date plus the model's claimable months equals its stated date.
- Fact extraction accuracy (C): PhD date correct; interruption types and durations correct (primary-carer periods summed; parental leave inside a primary-carer period expected as its own entry); attributes correct, including no unsupported flags.
- Replicability (C): re-running the rule engine on the logged facts reproduces the logged decision, date and trace exactly.
- Error category: assigned to every prediction that disagrees with ground truth, distinguishing extraction errors from rule-application errors.

6. Results

All 60 calls completed, with no API, parsing or schema failures. Architectures A and B were correct on all 20 cases for both eligibility and effective date; Architecture C was correct on 19 (Table 2, Figure 2). Every A and B output was internally consistent, and B's claimable-month count matched the ground truth in all 20 cases. C's extractor recovered every PhD date and every interruption type and duration, including the relocation cap cases, the concurrent position and nested parental leave; one attribute was wrong.

Architecture	Eligibility accuracy	Effective-date accuracy	Schema/output validity	Decision repayable from log	Auditability
A: LLM only	20/20 (1.00)	20/20 (1.00)	20/20 decision lines parsed	No	Free-text explanation only; decision not independently recomputable

Architecture	Eligibility accuracy	Effective-date accuracy	Schema/output validity	Decision repayable from log	Auditability
B: LLM + structured validation	20/20 (1.00)	20/20 (1.00)	20/20 schema-valid	No	Typed fields allow consistency checks; reasoning remains free text
C: LLM extraction + deterministic rules	19/20 (0.95)	19/20 (0.95)	20/20 schema-valid	20/20	Logged facts + rule trace; errors localisable to a field or rule step

Table 2. Architecture comparison (single run, n = 20 per architecture).

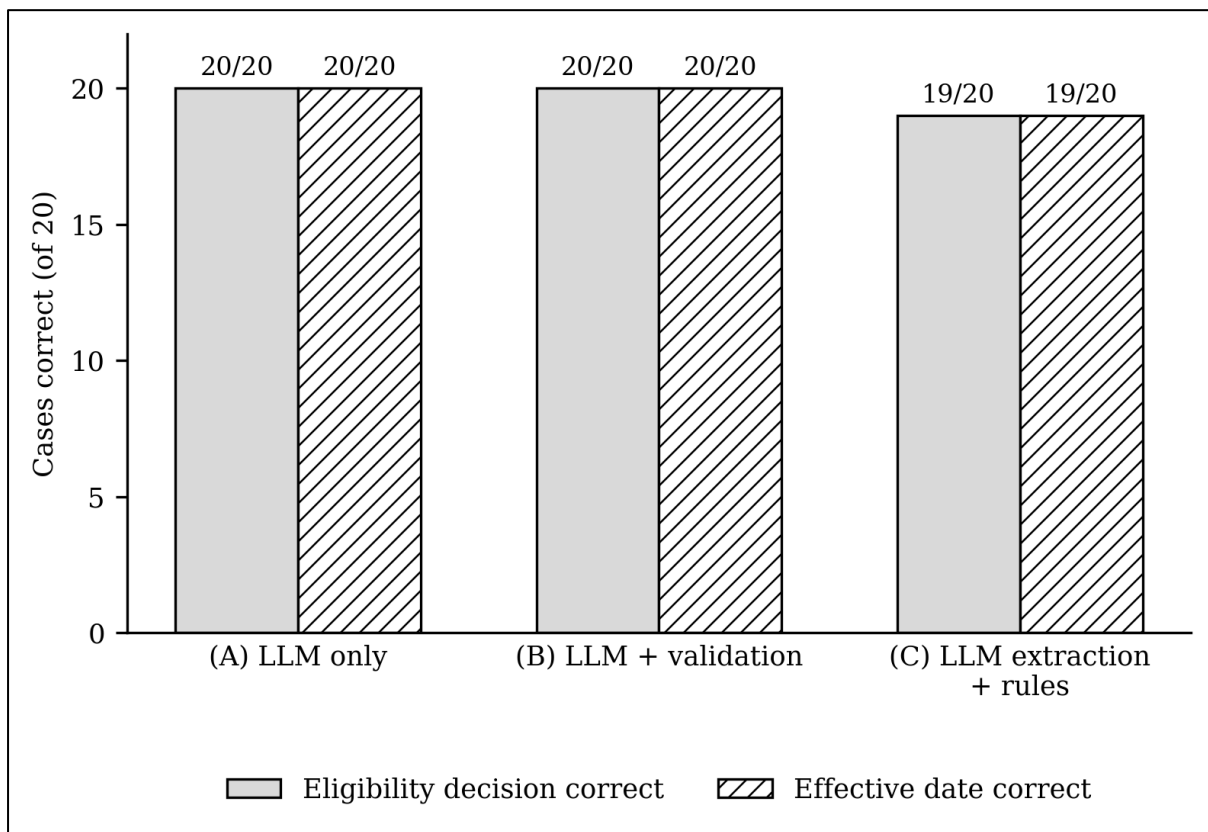


Figure 2. Correct eligibility decisions and effective dates by architecture (counts out of 20; single run; no significance testing).

With n = 20 and one run, the one-case difference between C and the other architectures is too small to support a comparative accuracy claim.

Figure 3 breaks each architecture's pipeline into its checkable stages. Of 220 stage-level checks, three failed, all for applicant A10 in Architecture C. No case failed in more than one architecture, and across all 60 predictions eligibility and effective-date errors always occurred together (59 both correct, 1 both incorrect).

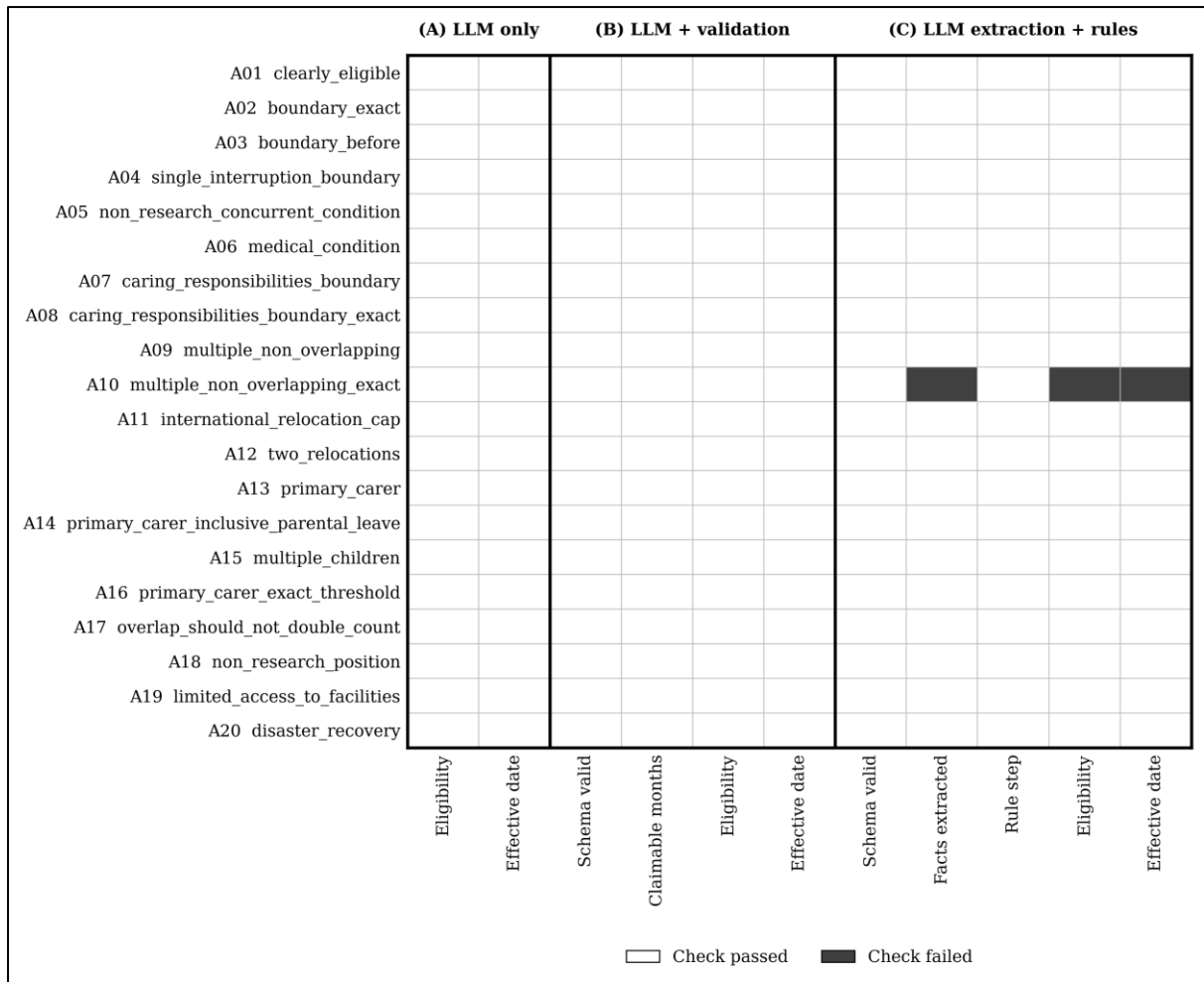


Figure 3. Error-pattern matrix. Each cell is one check for one applicant. The only failure began at fact extraction in C for A10 and propagated to the decision and date; C's rule step was correct for all 20 inputs.

A10's profile states seven months of parental leave and twelve months of non-concurrent non-research work, with no mention of a primary-carer period. The expected calculation was:

7 months parental leave + 12 months non-research position
 = 19 claimable months
 15 August 2019 + 19 months = 15 March 2021 -> ELIGIBLE

Architectures A and B reached this result. In C, the extractor reported both periods with correct types and durations, but set an attribute the profile does not support on the parental leave, while producing no primary-carer entry:

```
{
  "type": "parental_leave",
  "duration_months": 7,
  "after_phd": true,
  "within_primary_carer_period": true
}
```

The rule engine's logged trace shows how that fact was used:

PhD award date: 2019-08-15
 Interruption 1 (parental_leave, 7 months stated):

```

parental leave within primary-carer period; not added -> 0 claimable
Interruption 2 (non_research_position, 12 months stated):
allowable; counted in full -> 12 claimable
Total claimable months: 12
Effective date: 2019-08-15 + 12 months = 2020-08-15
2020-08-15 < threshold 2021-03-01: INELIGIBLE

```

Every step is the correct application of the rule to the extracted input. Re-running the engine on the logged facts reproduced the result exactly; replaying it with that one field corrected yields 19 months, 2021-03-15 and eligible, and with correct facts the engine reproduced all 20 expected outcomes. The error is therefore classified as an **LLM extraction error (an unsupported attribute)**, not a rule-application error. The deterministic layer did not correct the incorrect fact; it faithfully propagated it. The pipeline also reported A10 with status ok: no runtime check flagged the error, which was found only by comparison with ground truth.

7. Discussion

The architectures differed by only one case out of twenty, so the results do not establish an accuracy advantage for any architecture. A and B reached perfect accuracy, correctly handling caps, concurrency conditions and nested parental leave, so H1 was not supported in this run. B also produced valid and internally consistent JSON in every case, but because it made no errors, the experiment could not demonstrate a schema-valid but incorrect decision; H2 was therefore only partially observable.

The A10 case highlights the difference between extraction and deterministic execution. The rule engine applied every rule correctly but acted on an unsupported fact extracted by the LLM, removing seven months and reversing the decision. This shows that deterministic execution can prevent rule-application errors without eliminating errors in the extracted facts.

The main difference between the architectures was auditability. A and B provide outputs that can be inspected, but their decisions cannot be independently recomputed from explicit intermediate facts. C allowed all 20 decisions to be replayed exactly, and the A10 error was localised to a specific field and rule branch. However, traceability is not detection: C did not identify the error at runtime, and the experiment did not test how errors in A or B would be localised.

Within its boundaries, the experiment supports a small set of descriptive observations, and it is equally important to state what it does not show.

Supported by this run	Not established by this run
A and B achieved 20/20 eligibility and effective-date accuracy; C achieved 19/20	That any architecture is generally superior, or that A or B is more accurate than C
B and C produced 20/20 schema-valid outputs	That deterministic architectures are inherently more accurate
The only observed failure occurred in C and originated in LLM fact extraction	That the findings generalise to other LLMs, prompts, temperatures or providers
The rule engine correctly applied the incorrect extracted fact	That the systems are suitable for production research administration
All C decisions were replayable from logged facts	That the experiment represents real ARC assessments
C's A10 error could be localised to one field and one rule branch	How A or B errors would behave, since neither produced an error
The extractor behaved differently on near-identical profiles (A09, A10)	Run-to-run stability, or statistical generalisation beyond the 20 cases

Table 3. Scope of the findings.

9. Limitations

1. Small dataset: Only 20 synthetic cases were tested, so one error changes accuracy by five percentage points.
2. Synthetic ground truth: Cases and labels were constructed from the simplified rule rather than real applications.
3. Limited testing: Only one model, provider, prompt and run at temperature 0 were used.
4. Simplified rule: The experiment does not represent the full ARC policy.
5. Ceiling effect: A and B achieved 20/20, limiting meaningful accuracy comparison.
6. No human or statistical comparison: Human assessors and statistical significance were not tested.
7. LLM-dependent extraction: C remains dependent on the accuracy of the model's extracted facts.
8. Researcher-defined design: The schema, rule engine and ground truth reflect one interpretation of the task.

11. Further Improvements and Implementations

The extensions below would address the study's limitations or test the hypotheses raised by its single error. None has been implemented or tested.

Improvement	Purpose	Why it matters
Evidence-linked extraction	Require supporting text for each extracted fact.	Could have exposed the unsupported A10 fact.
Consistency checks	Flag contradictory or unsupported extracted attributes.	Could detect errors before rule execution.
Human review	Escalate ambiguous or unsupported cases.	Adds a safety check before consequential decisions.
Larger, harder dataset	Test ambiguous, contradictory and boundary cases.	Addresses the accuracy ceiling in the current 20-case dataset.
Multiple runs and models	Measure stability and model dependence.	Separates architecture effects from model or run effects.
Domain-expert validation	Independently verify the rule and ground truth.	Reduces dependence on a single interpretation of the policy.

Table 4. Proposed further improvements and implementations.

The A10 result raises the question future work should centre on: *can the system detect an incorrect extracted fact before it causes an incorrect decision?* A fuller evaluation of hybrid LLM-deterministic systems would measure extraction accuracy, error detection at runtime, error localisation, successful correction, final decision accuracy

12. Conclusion

This exploratory study compared three allocations of decision-making between an LLM and deterministic software on a research-funding eligibility task. This study found minimal accuracy difference between the three architectures: A and B achieved 20/20, while C achieved 19/20. The

main difference was auditability: C allowed every decision to be replayed, and its single error traced to a specific unsupported fact.

The key insight is that a deterministic boundary does not necessarily make an AI system more accurate; it makes part of its behaviour explicit, reproducible and inspectable. However, traceability is not detection, the A10 error was discovered through evaluation rather than at runtime. Future work should therefore examine not only where the model stops, but how errors are detected, corrected and independently reconstructed.

13. References

1. Australian Research Council. *Eligibility and Career Interruptions Statement*. Canberra: ARC; 2024. <https://www.arc.gov.au/sites/default/files/2024-03/Eligibility%20and%20Career%20Interruptions%20Statement.pdf>
2. OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. arXiv:2508.10925; 2025.
3. Groq. *Supported Models* (documentation). <https://console.groq.com/docs/models> (accessed October 2026).
4. Turpin M, Michael J, Perez E, Bowman SR. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*; 2023.
5. Ji Z, Lee N, Frieske R, Yu T, Su D, Xu Y, Ishii E, Bang YJ, Madotto A, Fung P. Survey of hallucination in natural language generation. *ACM Computing Surveys* 55(12); 2023.
6. Parasuraman R, Sheridan TB, Wickens CD. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics, Part A* 30(3):286-297; 2000.
7. Rudin C. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence* 1:206-215; 2019.
8. Raji ID, Smart A, White RN, Mitchell M, Gebru T, Hutchinson B, Smith-Loud J, Theron D, Barnes P. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT* '20)*, 33-44; 2020.